

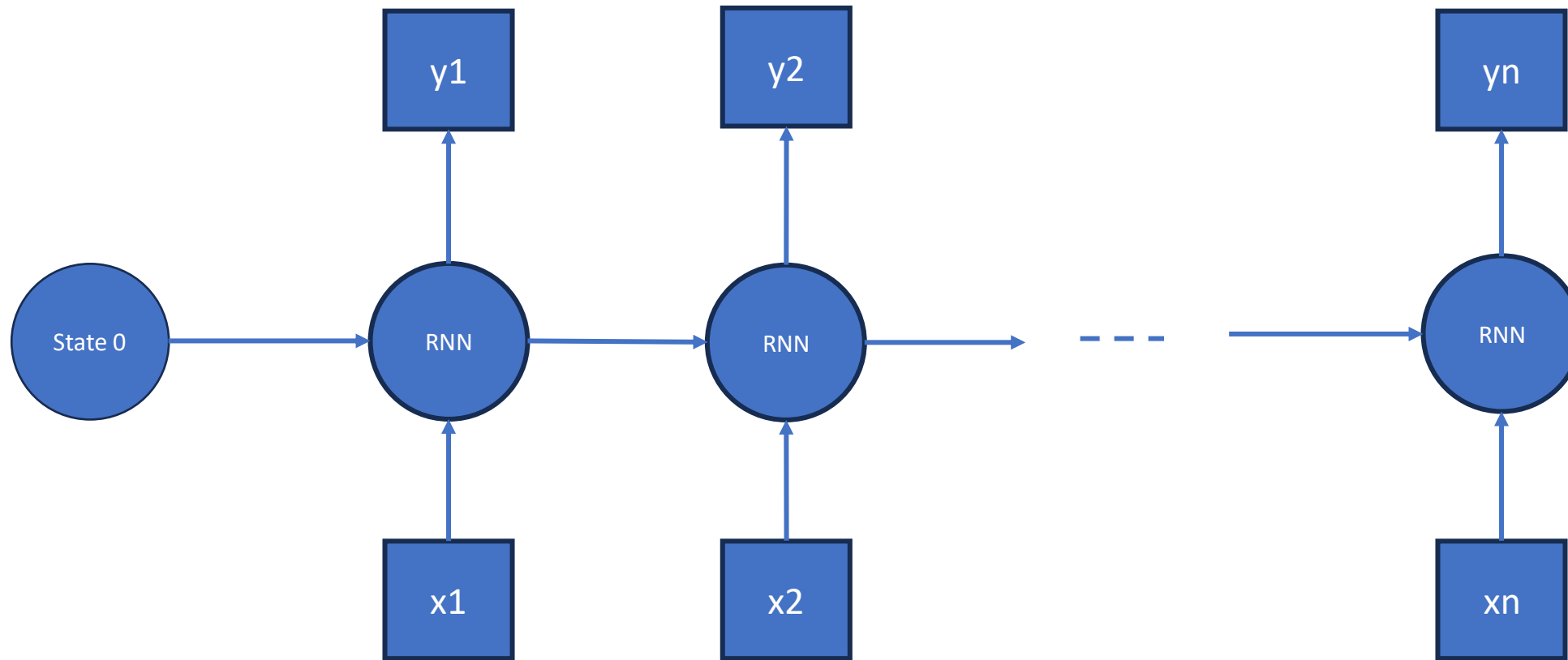
Transformers

Abu Bakr Soliman

Sequence to Sequence (Seq2Seq)

Input	x1	x2	..	xn
Output	y1	y2	..	yn

Recurrent Neural Network (RNN)

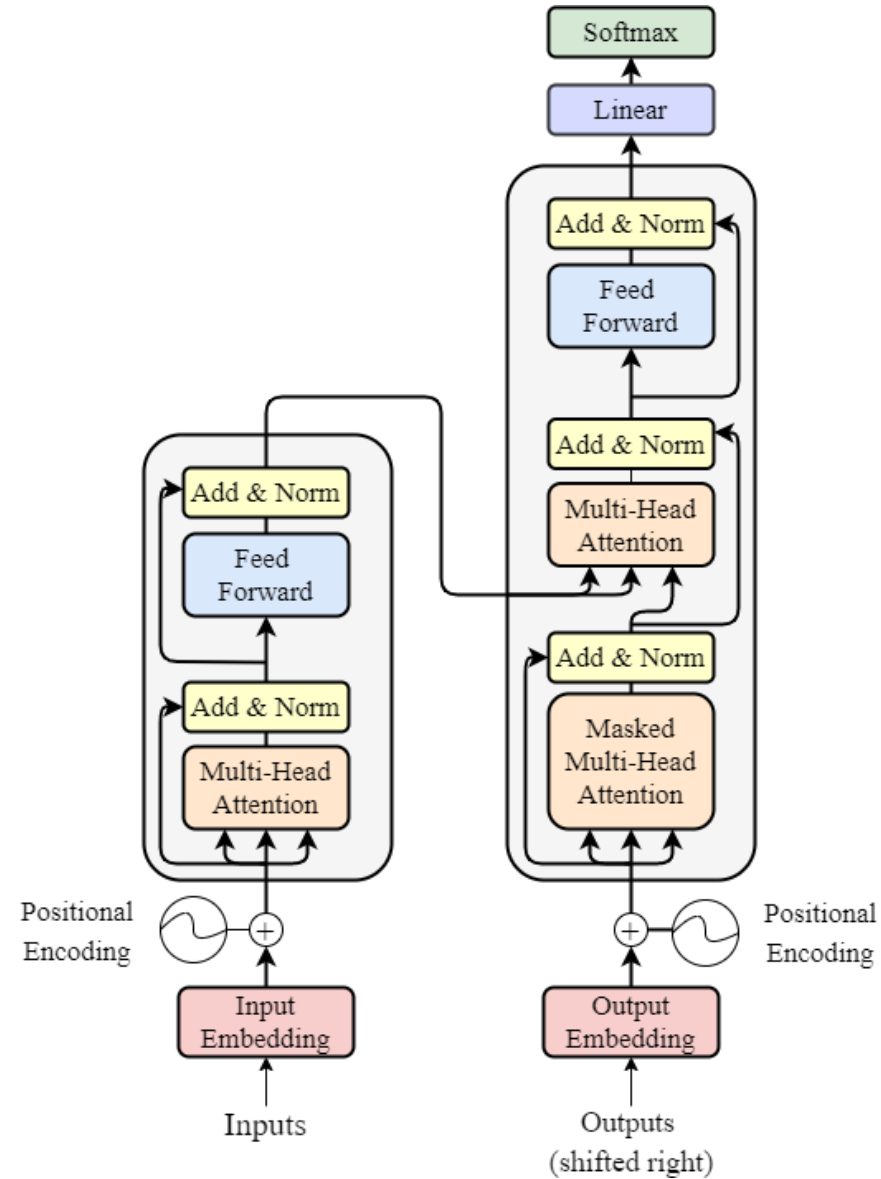


Time-Step

RNN Problems

- Long Sequences = Slow Computations
- Vanishing and Exploding Gradients
- Vanished Memory

Attention is All You Need



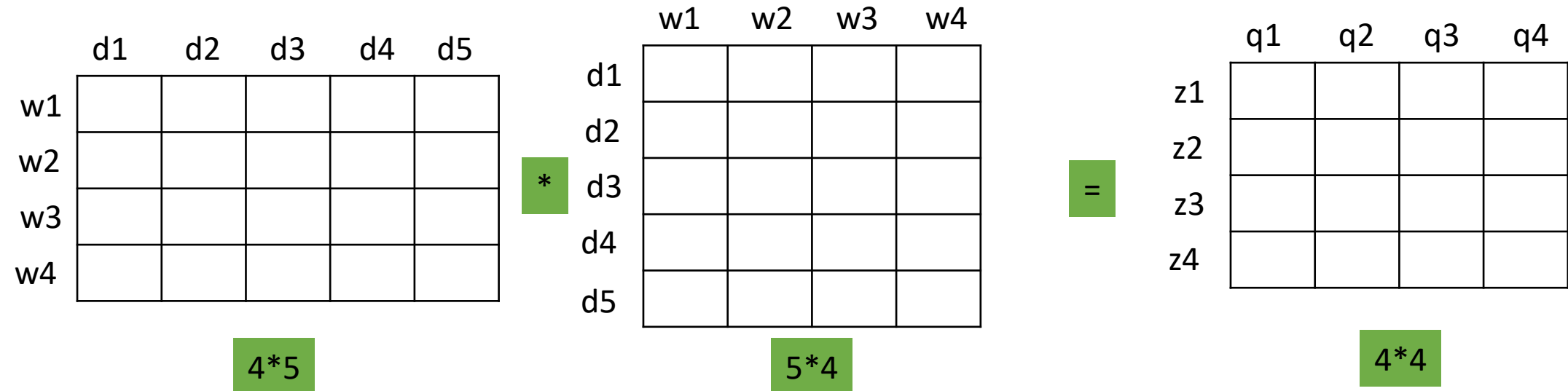
Vectors

Patient ID
10200
3600

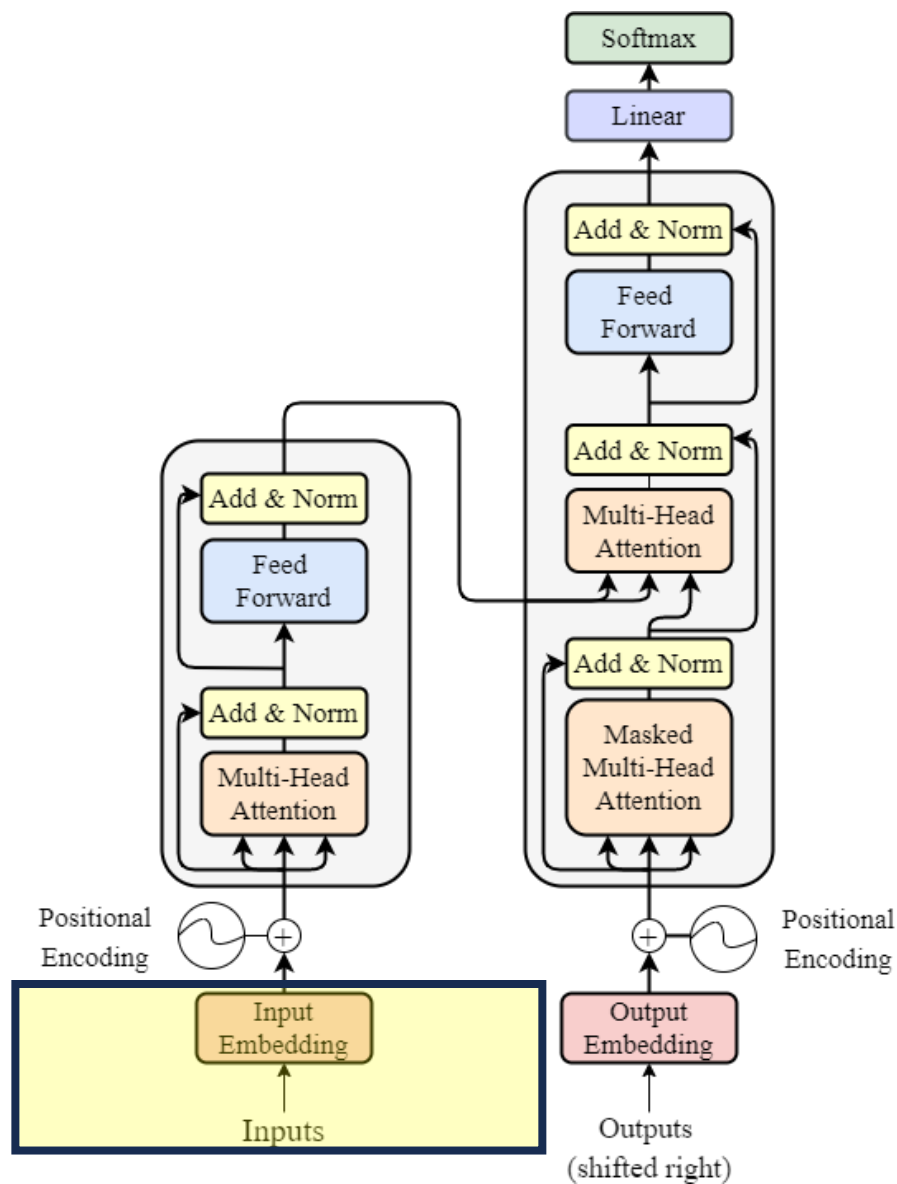
Vectors

Patient ID	Height (CM)	Weight (KG)	Systolic (mmHg)	Diastolic (mmHg)
10200	165	71	120	80
3600	180	92	110	85

Matrices Operations



Attention is All You Need



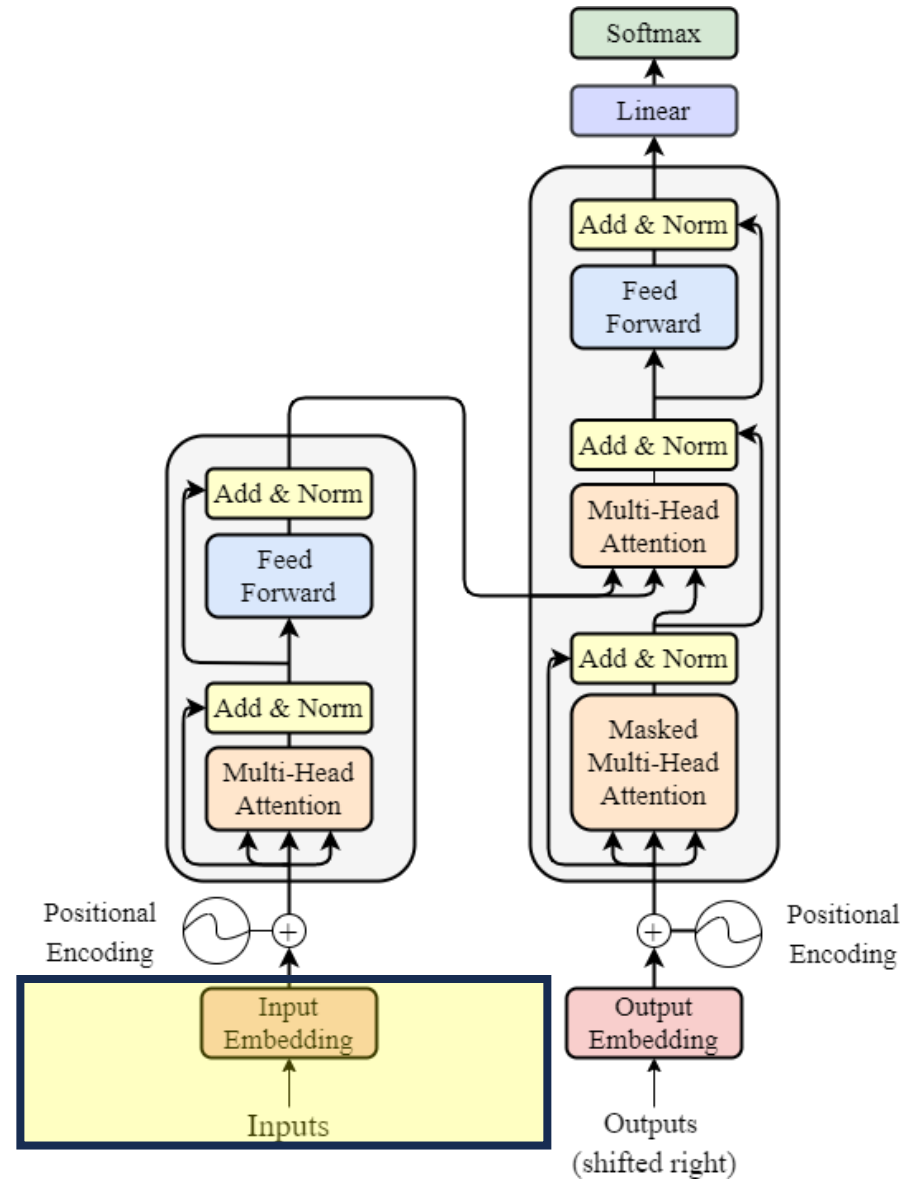
Tokenizer

Token	ID
Hello	1
Go	2
ed	3
..	..
red	32000

Input

I can go alone

	I	can	go	alone	Input Tokens
	105	255	1001	600	Input IDs
512	103.65 633.01 25.33 .. 152.06	636.22 2.01 96.25 .. 636.28	639.22 9.36 78.22 .. 96.35	986.22 7.22 9.36 .. 12.78	Embeddings



Input

512 {

512 {

512 {

I

105

103.65
..
152.06

+

..
..
..

=

103.65
..
152.06

can

255

636.22
..
636.28

+

..
..
..

=

636.22
..
636.28

go

1001

639.22
..
96.35

+

..
..
..

=

639.22
..
96.35

alone

600

986.22
..
12.78

+

..
..
..

=

986.22
..
12.78

Input Tokens

Input IDs

Embeddings

Positional Embeddings

Encoder Input

I

can

go

alone

Input Tokens

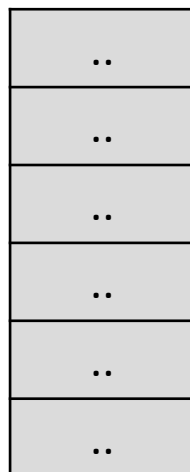
105

255

1001

600

Input IDs



$$PE(pos, 2i) = \sin \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

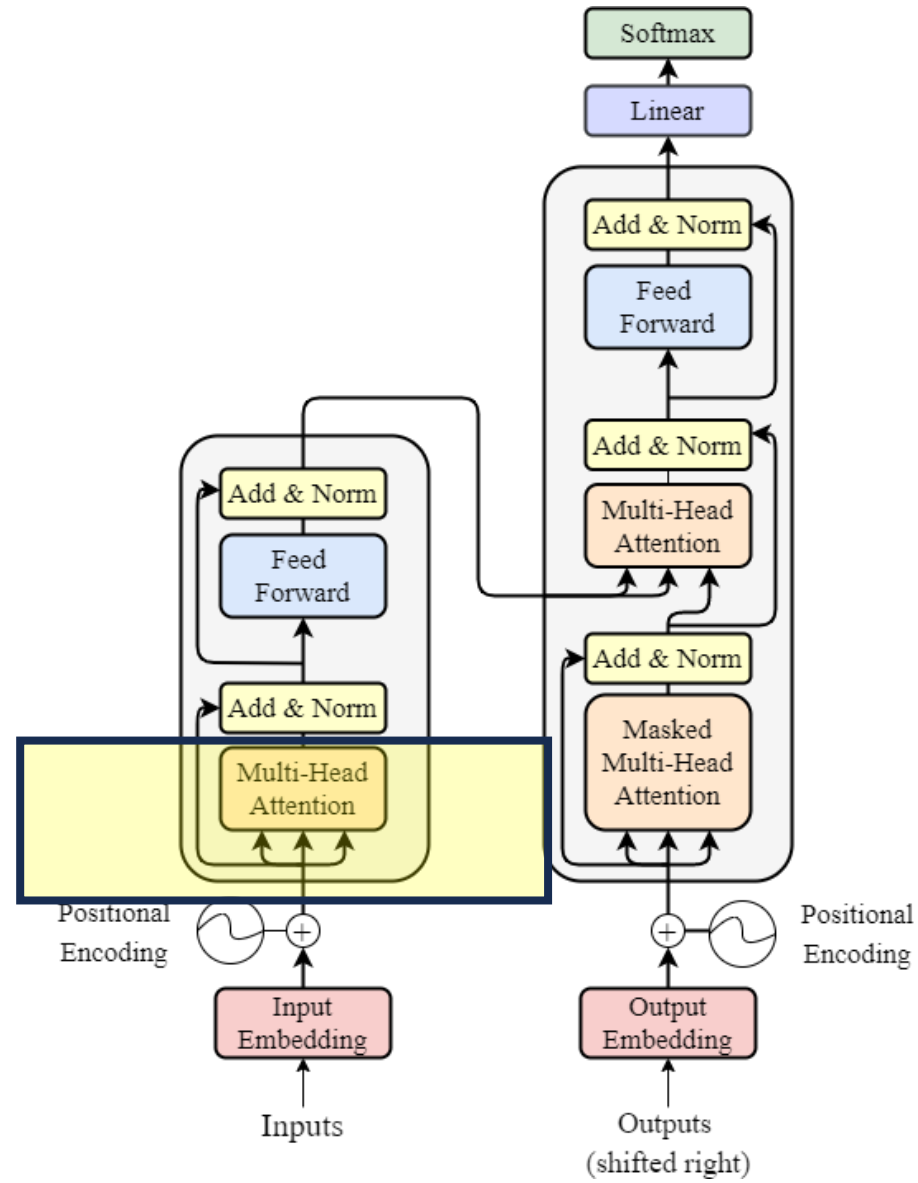
$$PE(pos, 2i + 1) = \cos \frac{pos}{10000^{\frac{2i}{d_{model}}}}$$

Run

```
pe = torch.zeros(512, 512)
```

```
for pos in range(512):  
    for i in range(256):
```

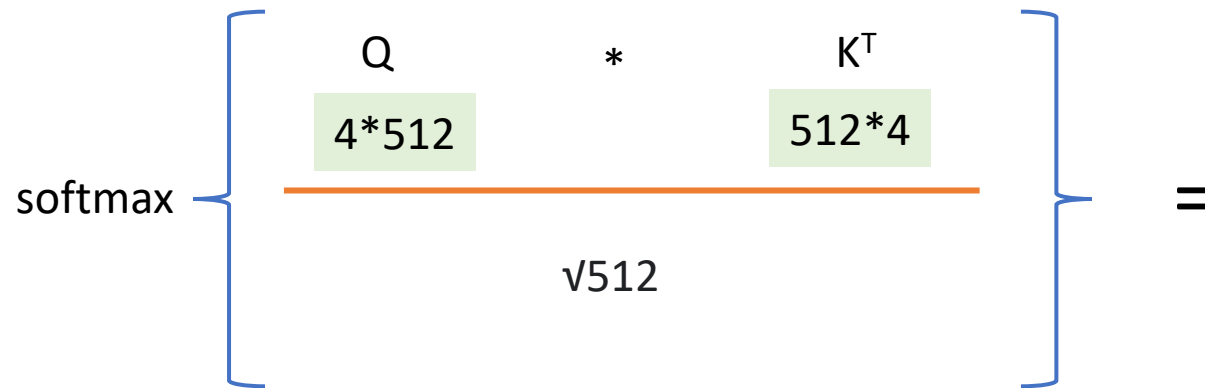
```
        pe[pos, 2*i] = sin(pos / (10000 ** (2*i/512)))  
        pe[pos, 2*i+1] = cos(pos / (10000 ** (2*i/512)))
```



Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$d_k = d_{\text{model}}$	512
seq	4



	I	can	go	alone
I	0.7	0.2	0.1	0.1
can	0.3	0.5	0.1	0.1
go	0.1	0.3	0.4	0.2
alone	0.05	0.05	0.1	0.8

4*4

Self-Attention

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

	I	can	go	alone
I	0.7	0.2	0.1	0.1
can	0.3	0.5	0.1	0.1
go	0.1	0.3	0.4	0.2
alone	0.05	0.05	0.1	0.8

4*4

*

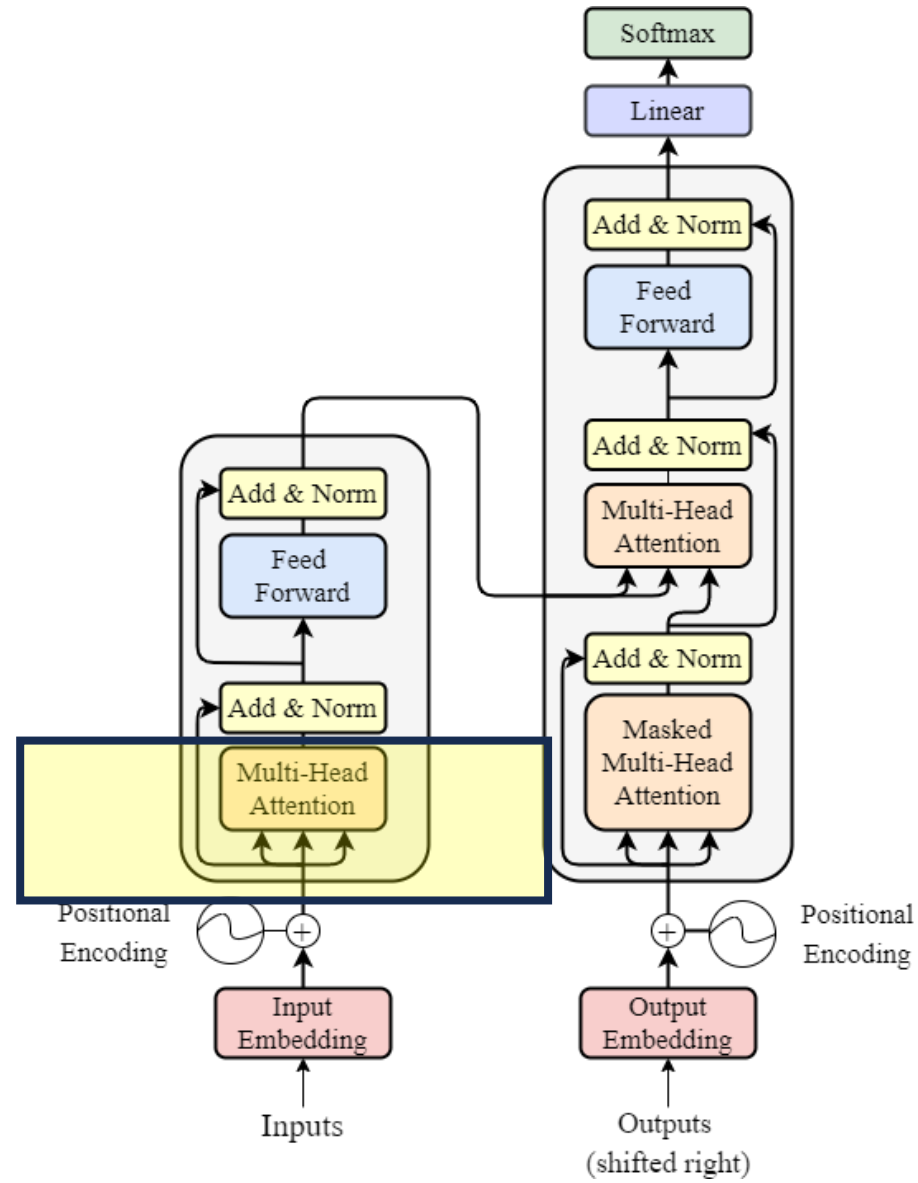
V

=

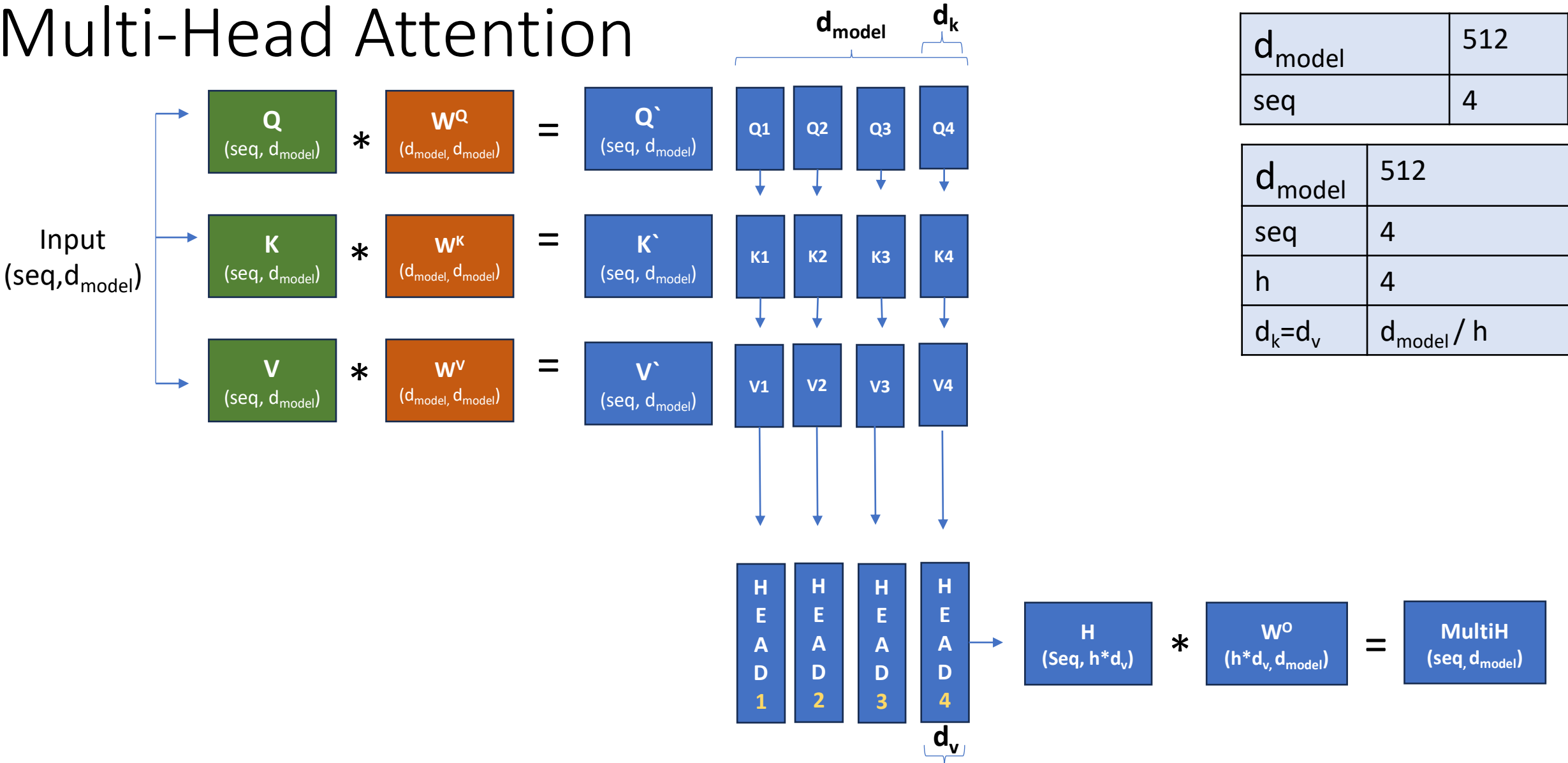
4*512

Attention(Q, K, V)

4*512

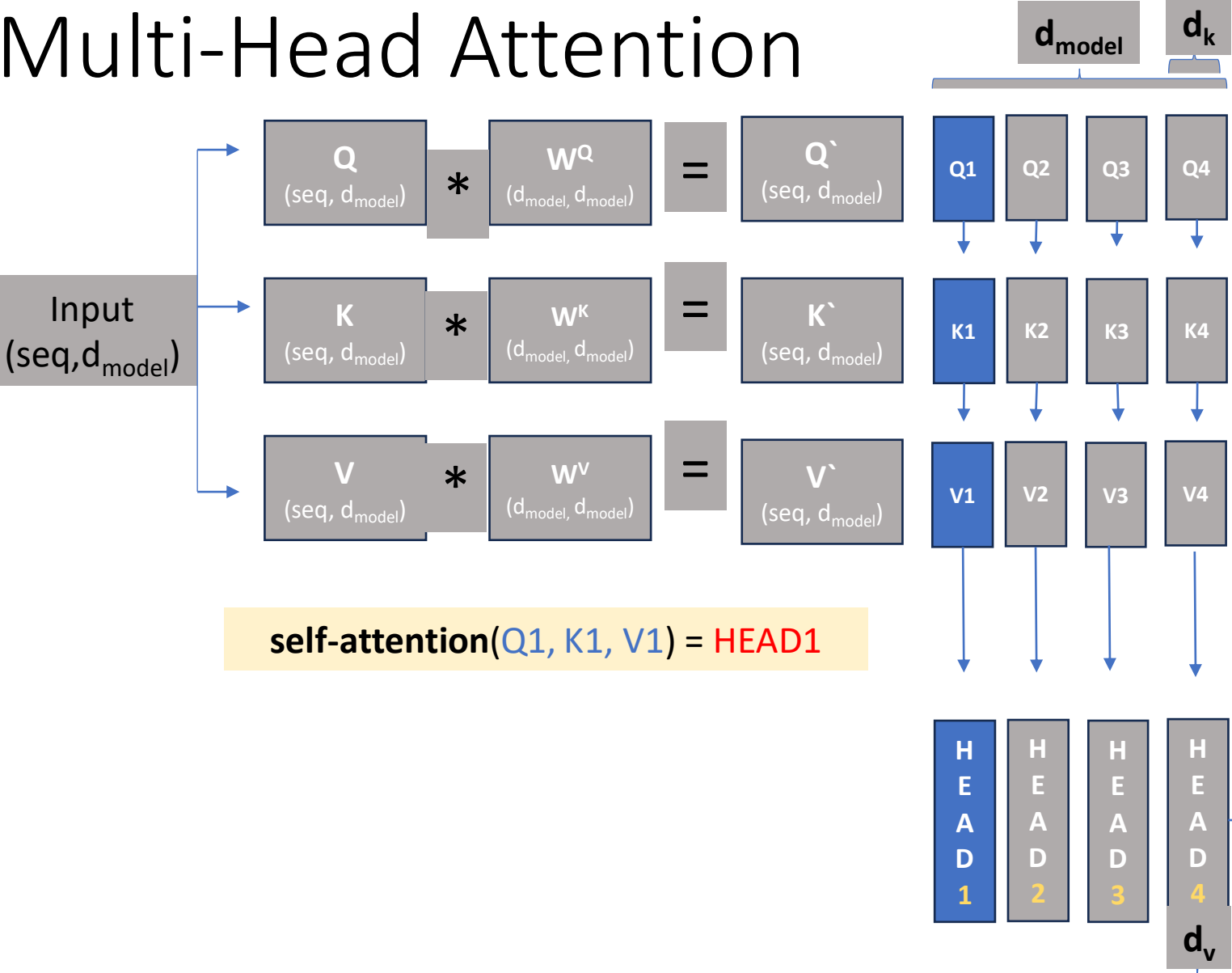


Multi-Head Attention



$MultiHead(Q, K, V) = Concat(head_1 \dots head_h)W^O$

Multi-Head Attention

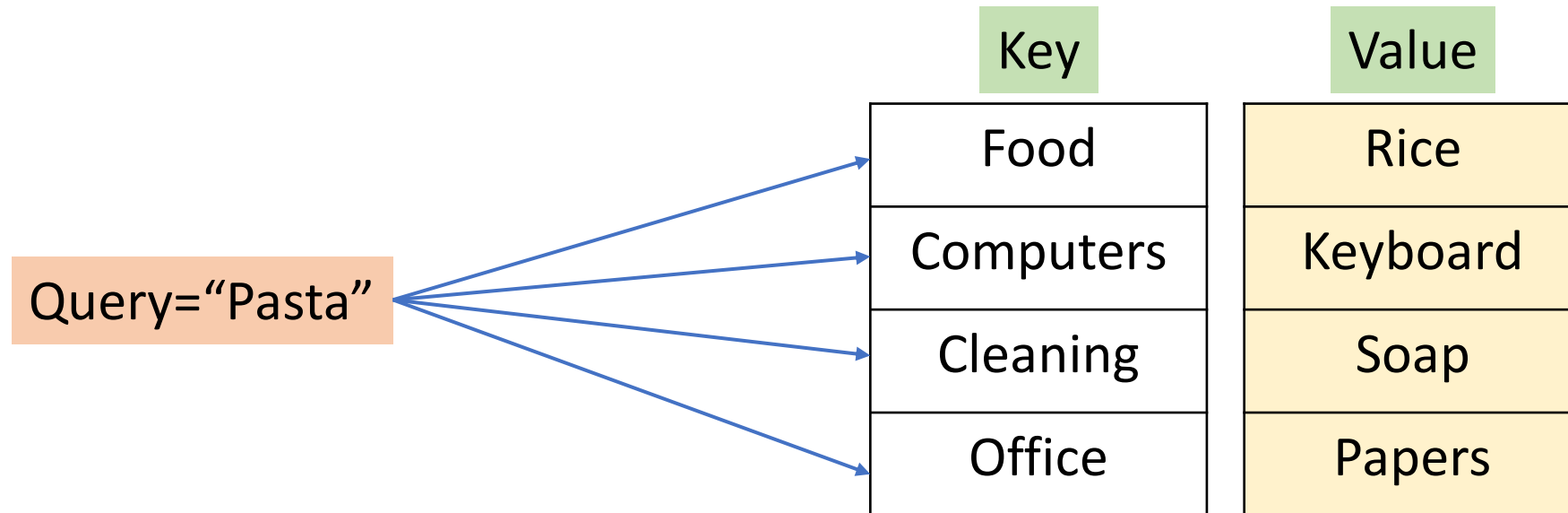


d _{model}	512
seq	4

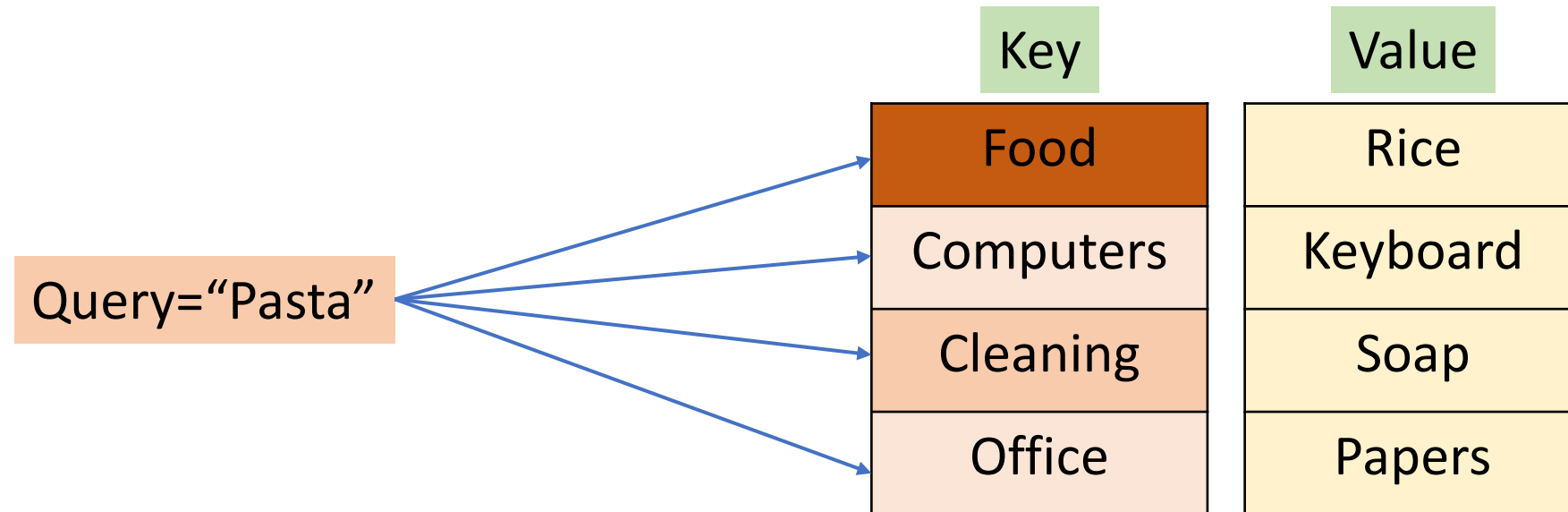
d _{model}	512
seq	4
h	4
d _k =d _v	d _{model} / h

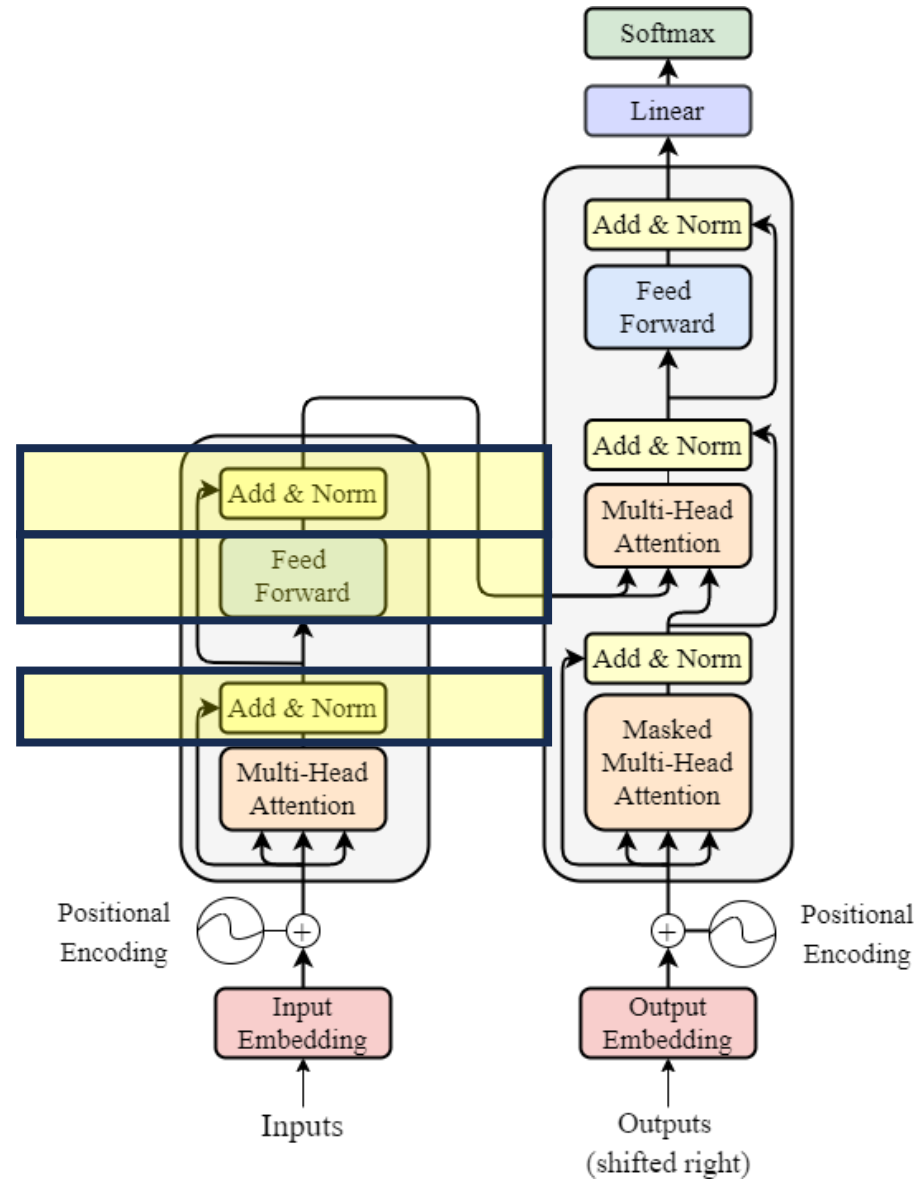
$MultiHead(Q, K, V) = Concat(head_1 \dots head_h)W^O$

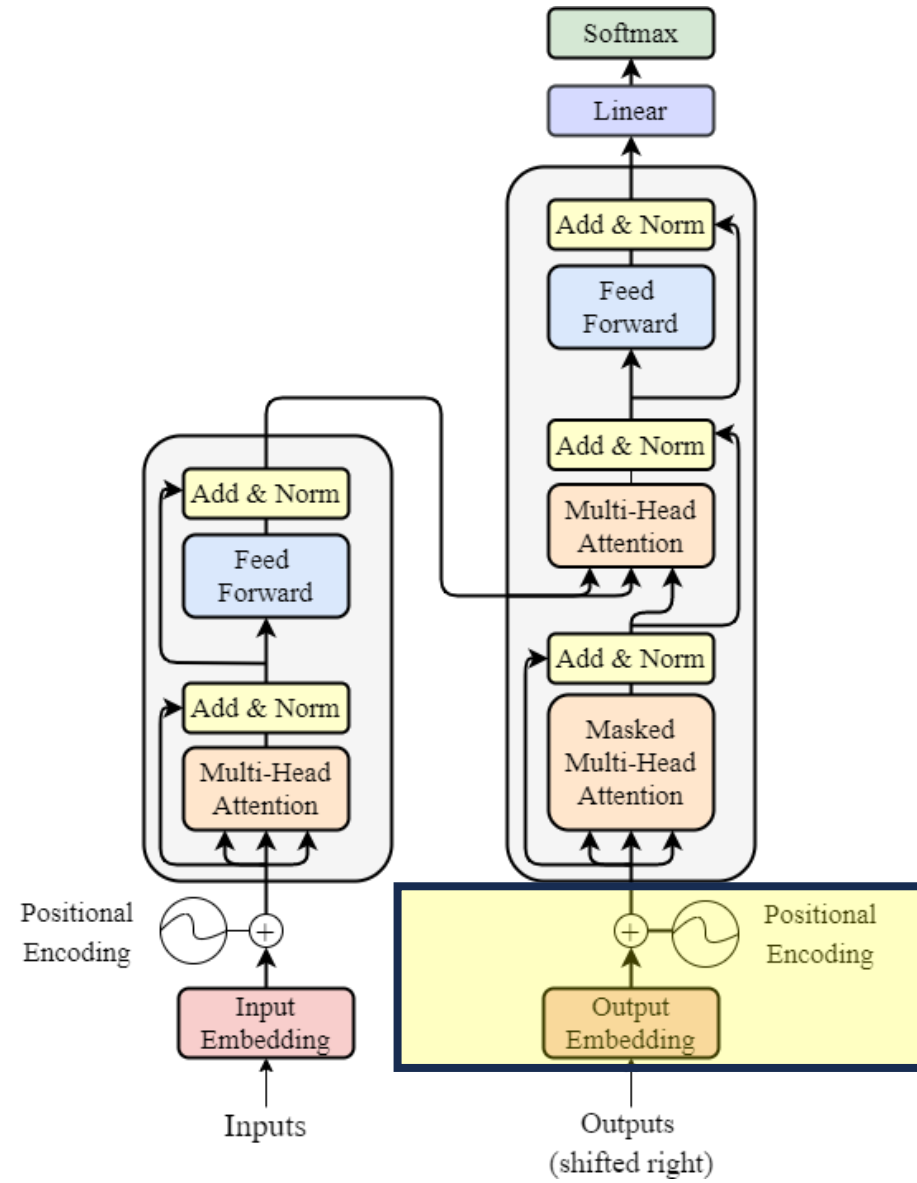
Query, Key & Value

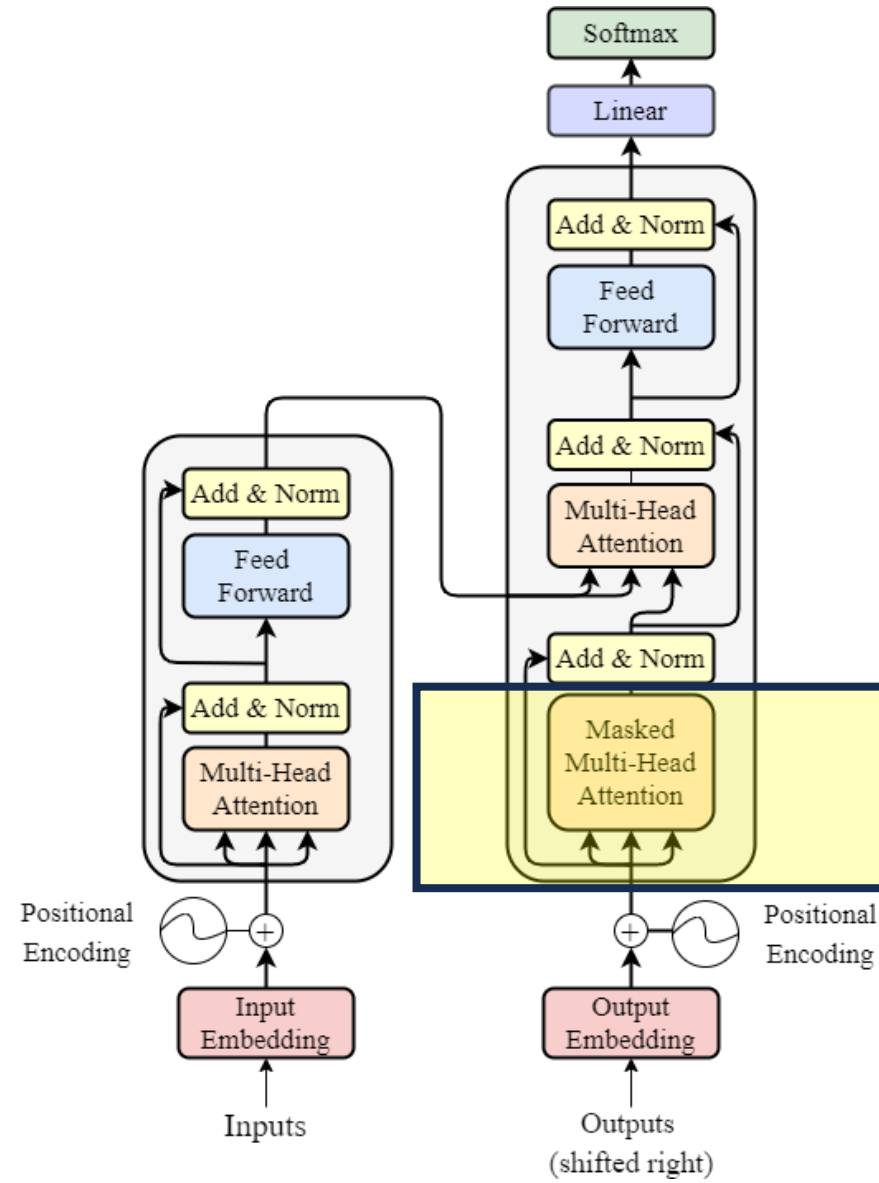


Query, Key & Value









Masked Attention

$$\text{softmax} \left\{ \frac{Q * K^T}{\sqrt{512}} \right\} =$$

	I	can	go	alone
I	0.7	0.2	0.1	0.1
can	0.3	0.5	0.1	0.1
go	0.1	0.3	0.4	0.2
alone	0.05	0.05	0.1	0.8

4*4

Masked Attention

Causal Model: The model must not be able to see the **future** words

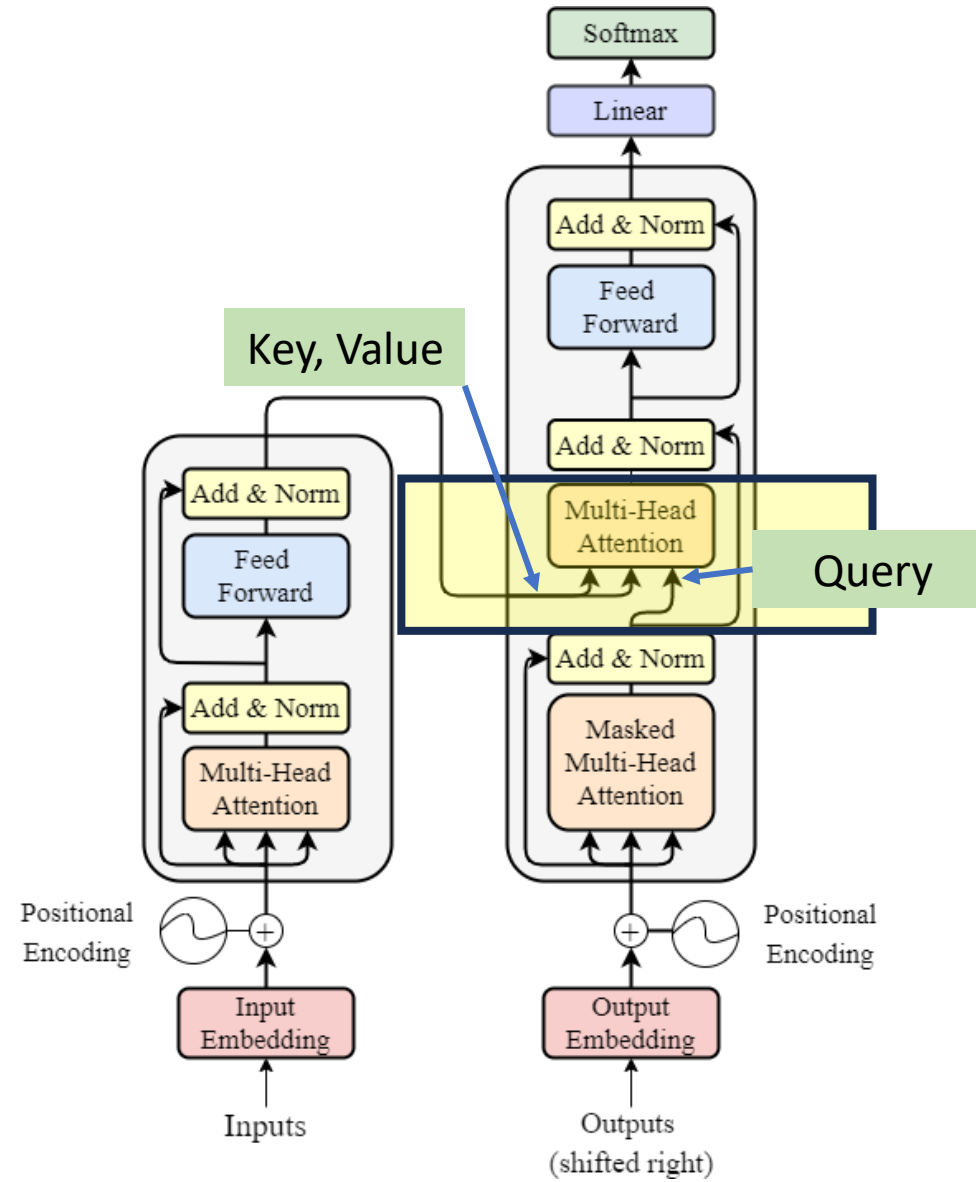
$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

$$\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

	I	can	go	alone
I	63.3	1.2	2.6	7.2
can	3.25	0.3	1.2	2.1
go	12.0	11.9	52.9	2.9
alone	1.6	63.1	14.2	101.3



	I	can	go	alone
I	63.3	$-\infty$	$-\infty$	$-\infty$
can	3.25	96.1	$-\infty$	$-\infty$
go	12.0	11.9	52.9	$-\infty$
alone	1.6	63.1	14.2	101.3



Training

I	can	go	alone
---	-----	----	-------



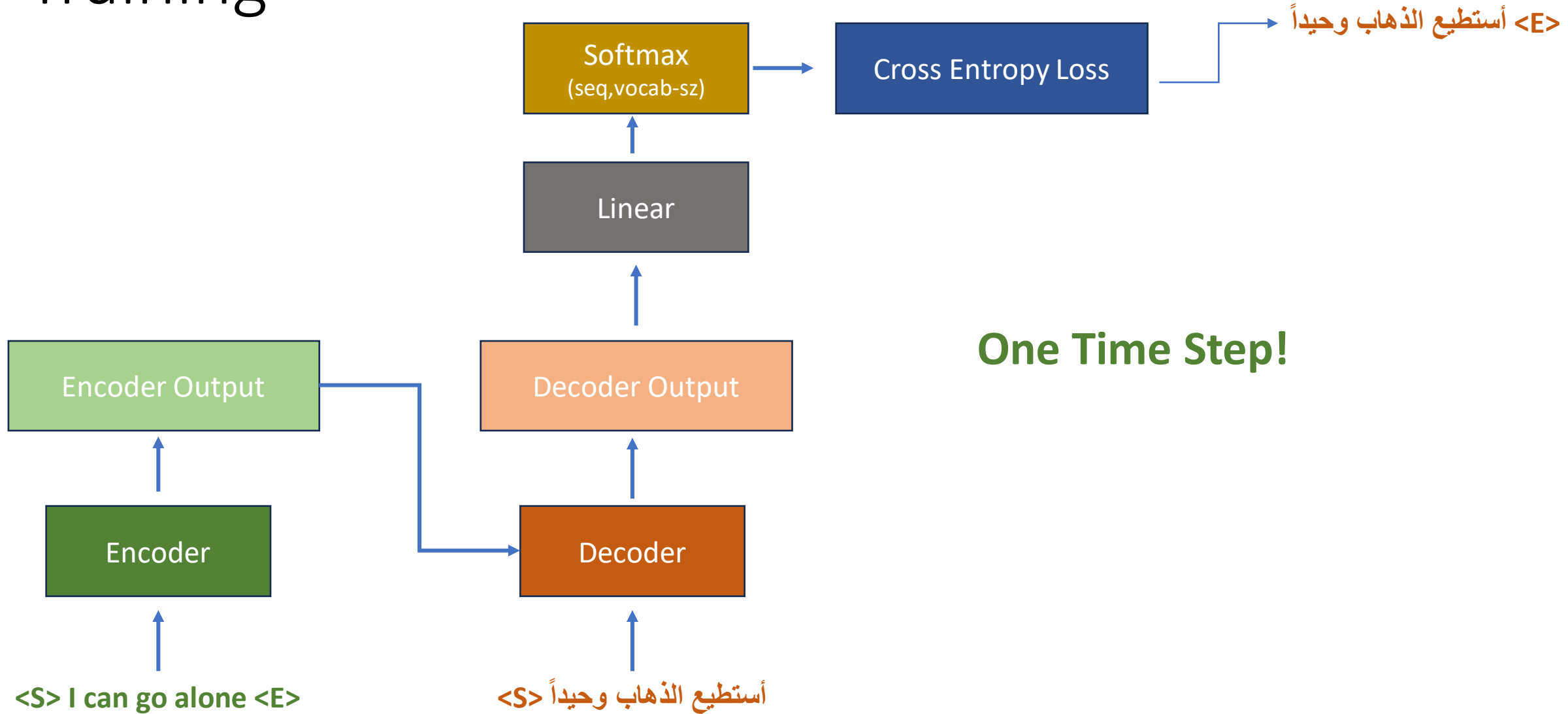
أستطيع	الذهاب	وحيداً
--------	--------	--------

<S>	I	can	go	alone	<E>
-----	---	-----	----	-------	-----



<S>	أستطيع	الذهاب	وحيداً	<E>
-----	--------	--------	--------	-----

Training



Inference

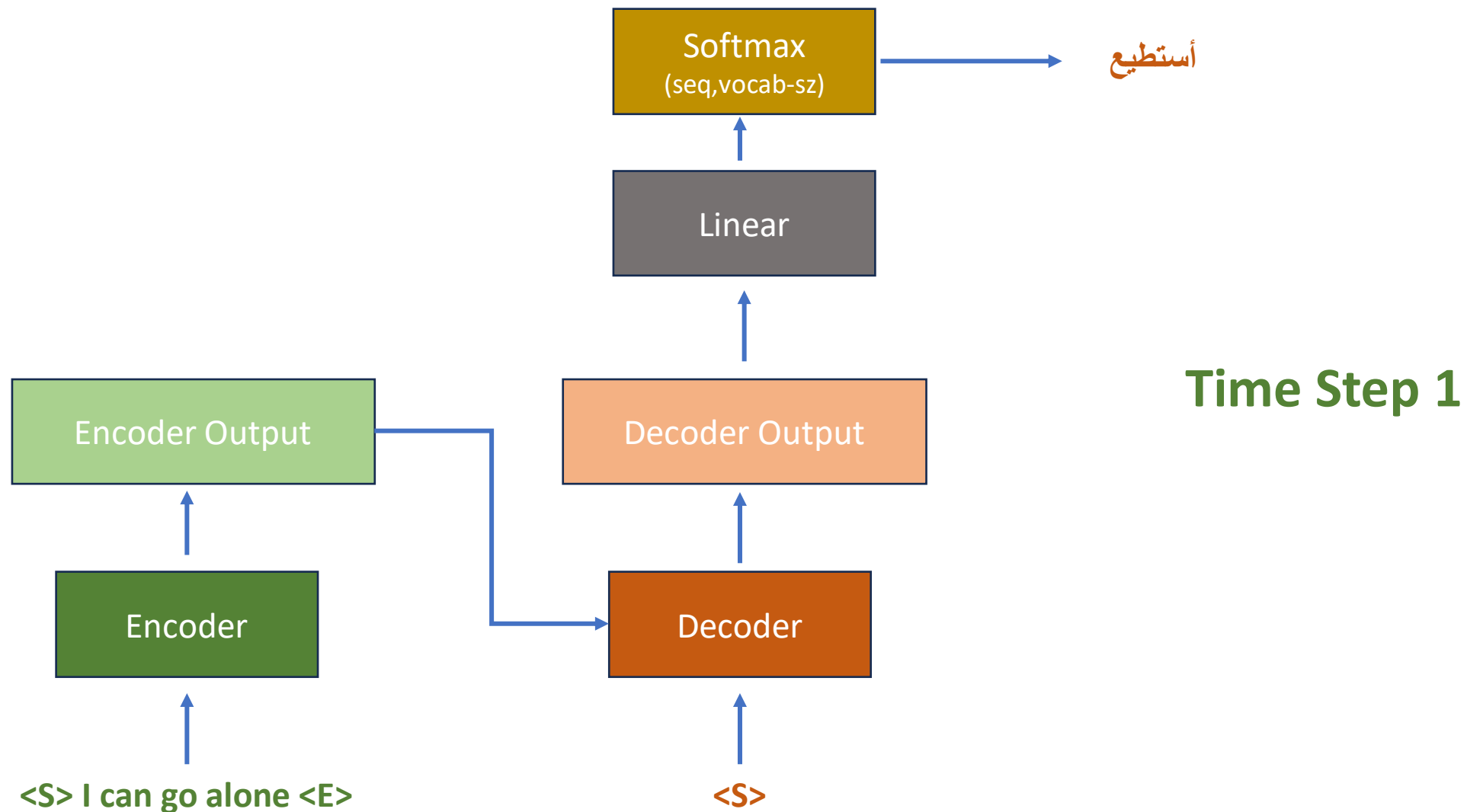
I	can	go	alone
---	-----	----	-------

<S>	I	can	go	alone	<E>
-----	---	-----	----	-------	-----

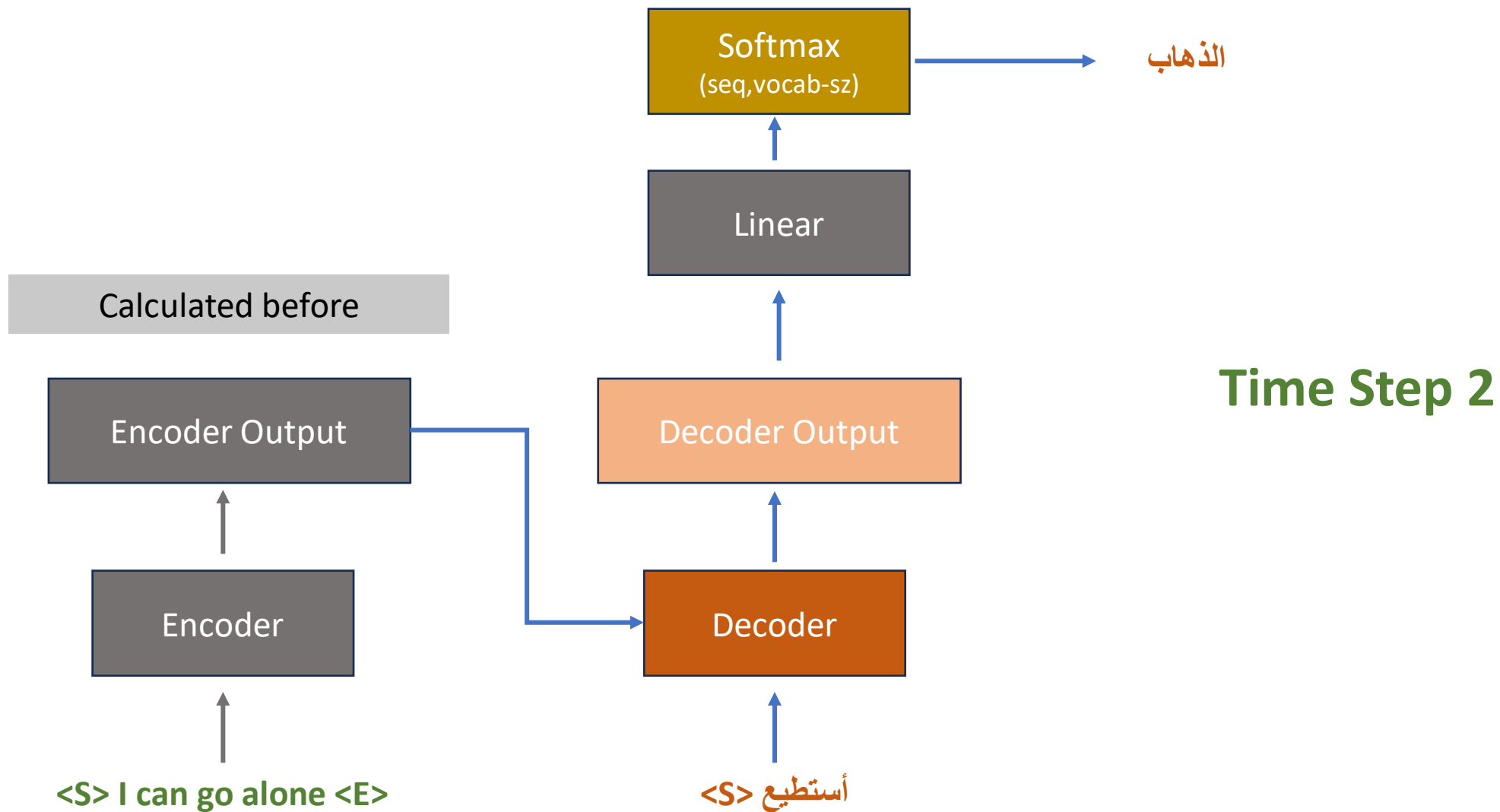


<S>

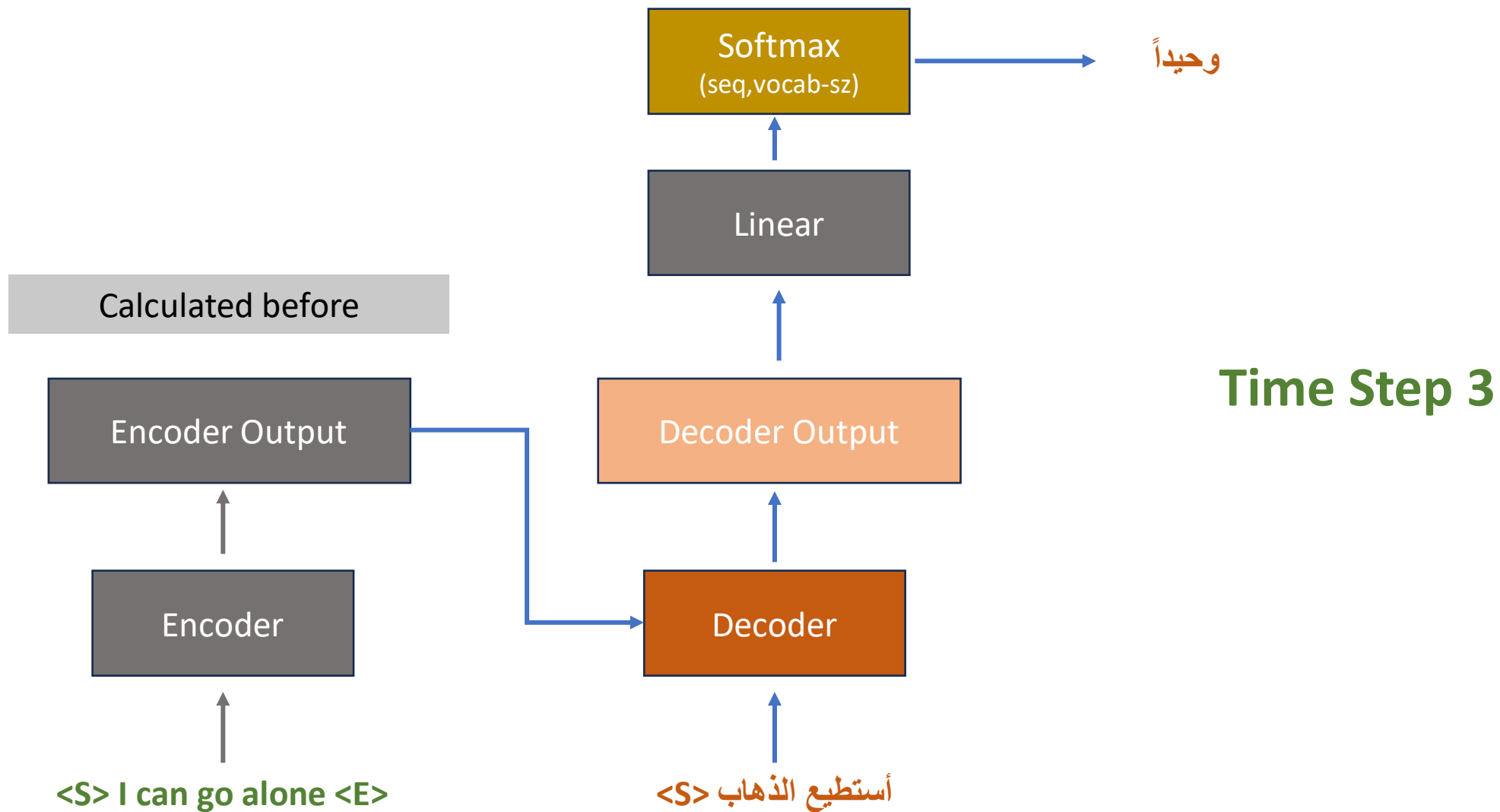
Inference



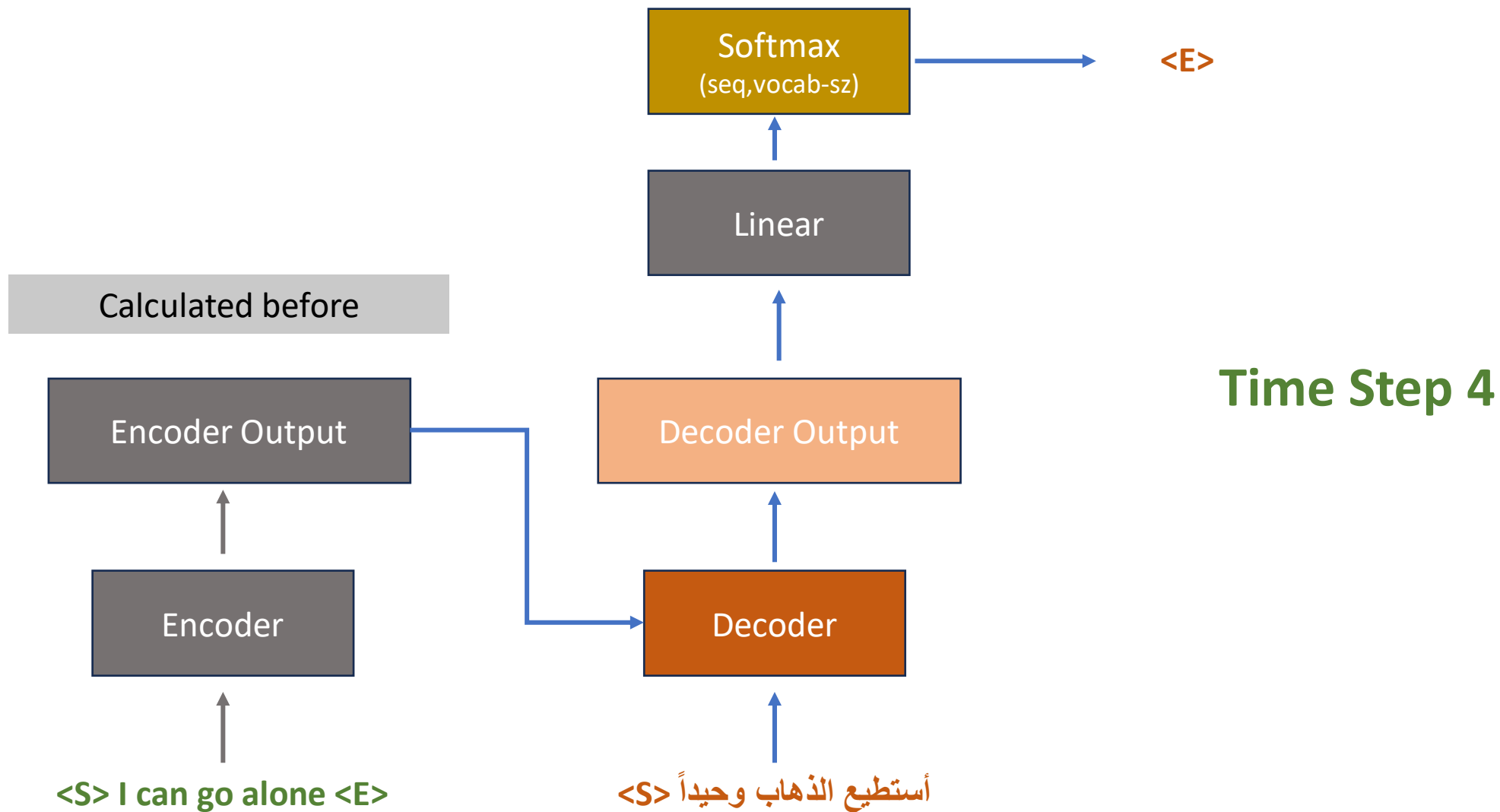
Inference



Inference



Inference



أستطيع الذهاب وحيداً <S>

How to generate text: using different decoding methods for language generation with Transformers

Published March 1, 2020

[Update on GitHub](#)



[patrickvonplaten](#)
[Patrick von Platen](#)

 [Open in Colab](#)